AI ASSISTED CODING

ASSIGNMENT-5.1 AND 6

NAME: Geethanjali

H.T.No.:2303A51005

Task 1:

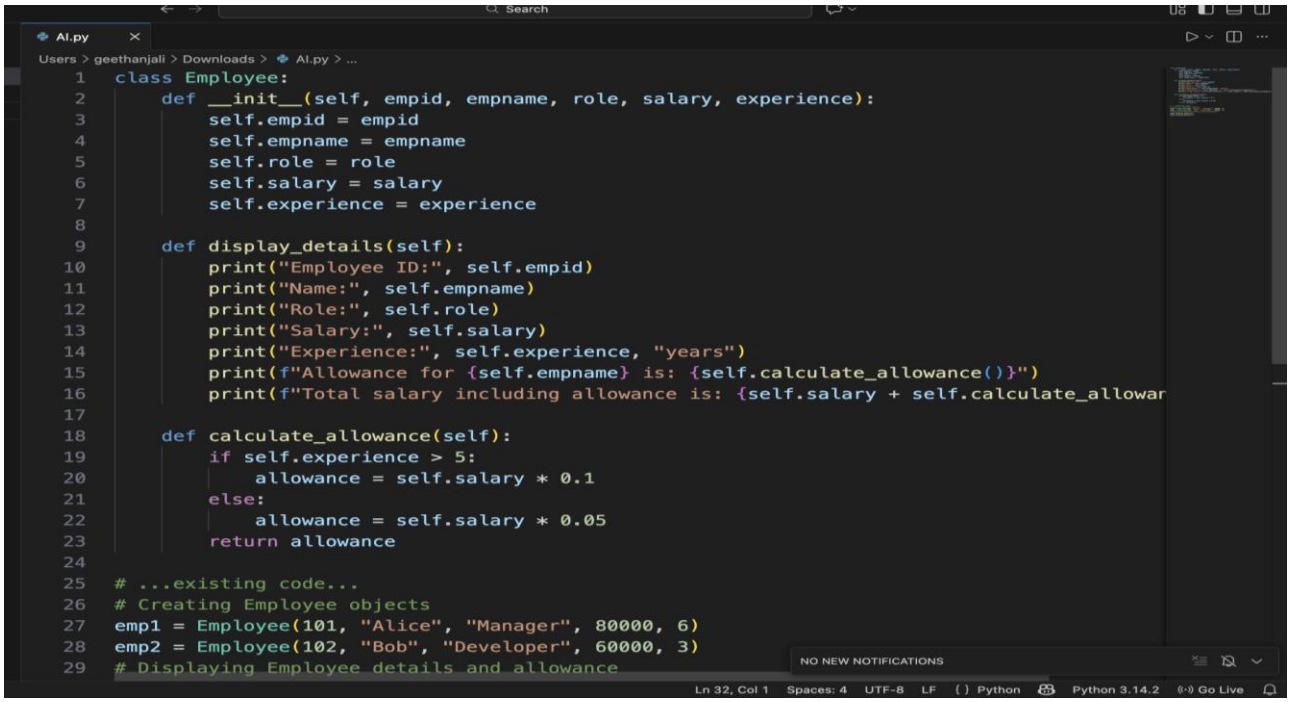
Employee Data: Create Python code that defines a class named `Employee` with the following attributes: `empid`, `empname`,`designation`, `basic_salary`, and `exp`. Implement a method `display_details()` to print all employee details. Implement another method `calculate_allowance()` to determine additional allowance based on experience:

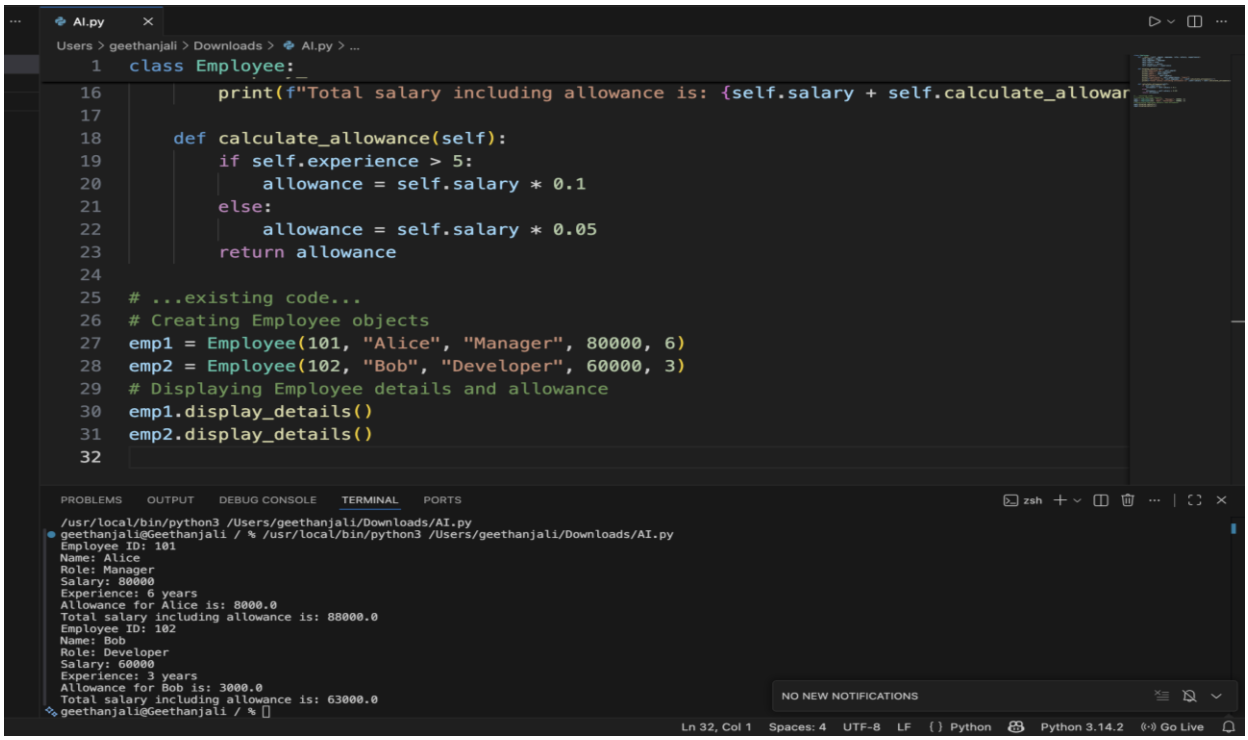
- If `exp > 10 years` → allowance = 20% of `basic_salary`

- If `5 ≤ exp ≤ 10 years` → allowance = 10% of `basic_salary`

- If `exp < 5 years` → allowance = 5% of `basic_salary`

Finally, create at least one instance of the `Employee` class, call the `display_details()` method, and print the calculated allowance.





Task 2:

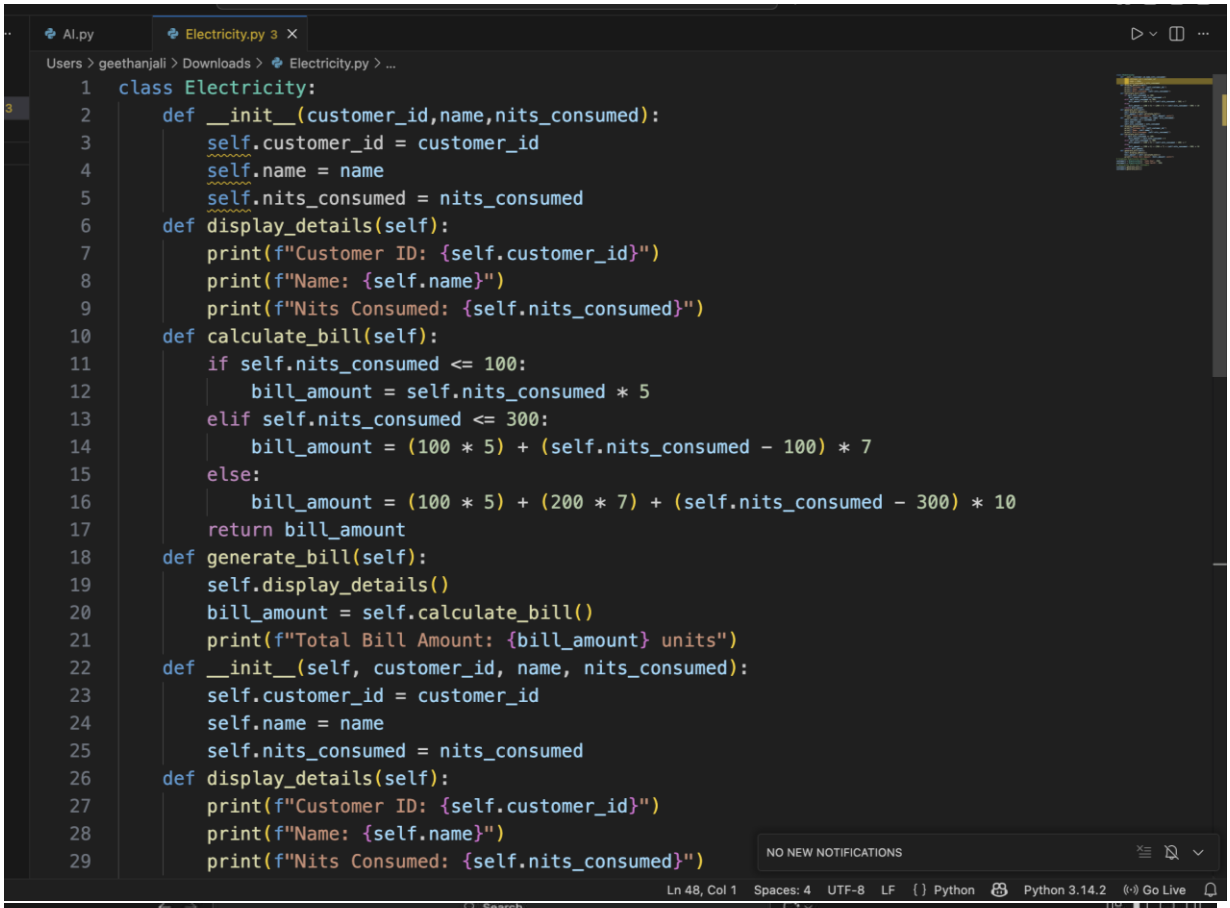
Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer_id`, `name`, and `units_consumed`. Implement a method `display_details()` to print customer details, and a method `calculate_bill()` where:

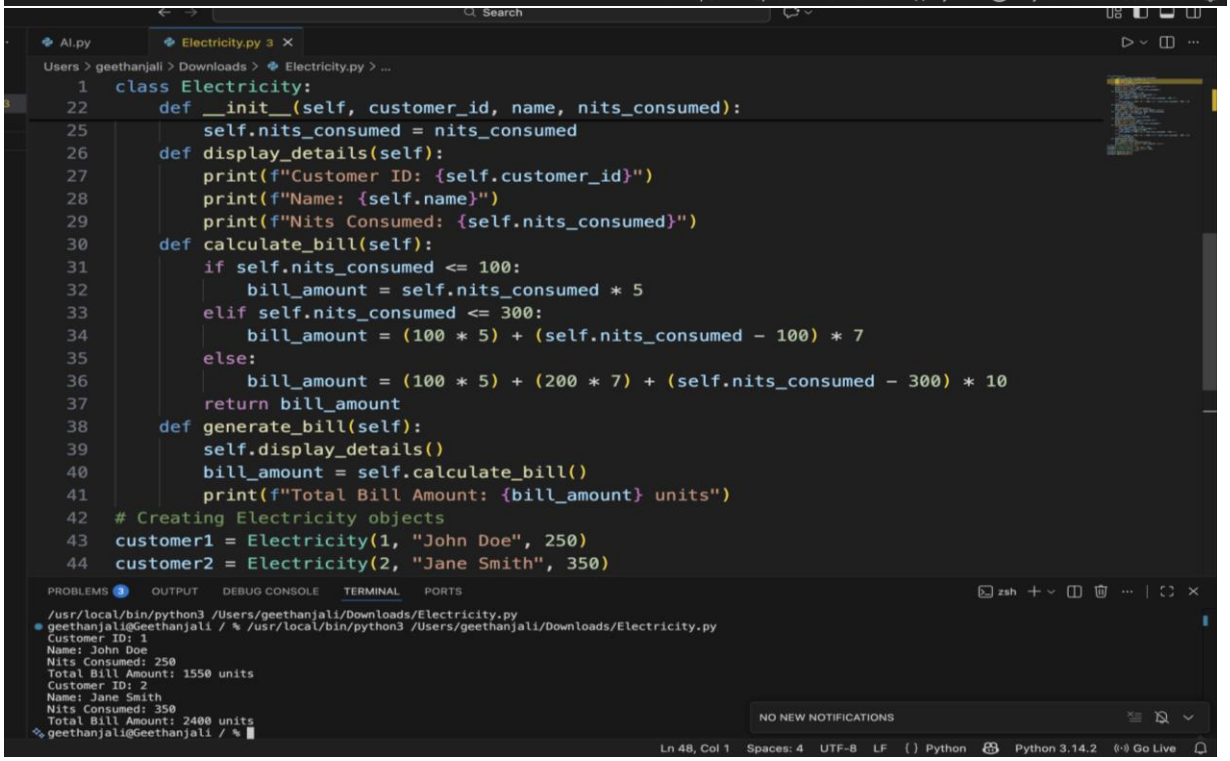
- Units ≤ 100 → ₹5 per unit

- 101 to 300 units → ₹7 per unit

- More than 300 units → ₹10 per unit

Create a bill object, display details, and print the total bill amount.


AI.py
Electricity.py 3
Users > geethanjali > Downloads > Electricity.py > ...
class Electricity:
def __init__(customer_id,name,nits_consumed):
self.customer_id = customer_id
self.name = name
self.nits_consumed = nits_consumed
def display_details(self):
print(f"Customer ID: {self.customer_id}")
print(f"Name: {self.name}")
print(f"Nits Consumed: {self.nits_consumed}")
def calculate_bill(self):
if self.nits_consumed <= 100:
bill_amount = self.nits_consumed * 5
elif self.nits_consumed <= 300:
bill_amount = (100 * 5) + (self.nits_consumed - 100) * 7
else:
bill_amount = (100 * 5) + (200 * 7) + (self.nits_consumed - 300) * 10
return bill_amount
def generate_bill(self):
self.display_details()
bill_amount = self.calculate_bill()
print(f"Total Bill Amount: {bill_amount} units")
def __init__(self, customer_id, name, nits_consumed):
self.customer_id = customer_id
self.name = name
self.nits_consumed = nits_consumed
def display_details(self):
print(f"Customer ID: {self.customer_id}")
print(f"Name: {self.name}")
print(f"Nits Consumed: {self.nits_consumed}")
NO NEW NOTIFICATIONS
Ln 48, Col 1 Spaces: 4 UTF-8 LF { } Python Python 3.14.2 Go Live


Search
AI.py
Electricity.py 3
Users > geethanjali > Downloads > Electricity.py > ...
class Electricity:
def __init__(self, customer_id, name, nits_consumed):
self.nits_consumed = nits_consumed
def display_details(self):
print(f"Customer ID: {self.customer_id}")
print(f"Name: {self.name}")
print(f"Nits Consumed: {self.nits_consumed}")
def calculate_bill(self):
if self.nits_consumed <= 100:
bill_amount = self.nits_consumed * 5
elif self.nits_consumed <= 300:
bill_amount = (100 * 5) + (self.nits_consumed - 100) * 7
else:
bill_amount = (100 * 5) + (200 * 7) + (self.nits_consumed - 300) * 10
return bill_amount
def generate_bill(self):
self.display_details()
bill_amount = self.calculate_bill()
print(f"Total Bill Amount: {bill_amount} units")
# Creating Electricity objects
customer1 = Electricity(1, "John Doe", 250)
customer2 = Electricity(2, "Jane Smith", 350)
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS
zsh
/usr/local/bin/python3 /Users/geethanjali/Downloads/Electricity.py
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/Electricity.py
Customer ID: 1
Name: John Doe
Nits Consumed: 250
Total Bill Amount: 1550 units
Customer ID: 2
Name: Jane Smith
Nits Consumed: 350
Total Bill Amount: 2400 units
geethanjali@Geethanjali / %
NO NEW NOTIFICATIONS
Ln 48, Col 1 Spaces: 4 UTF-8 LF { } Python Python 3.14.2 Go Live

Task 3:

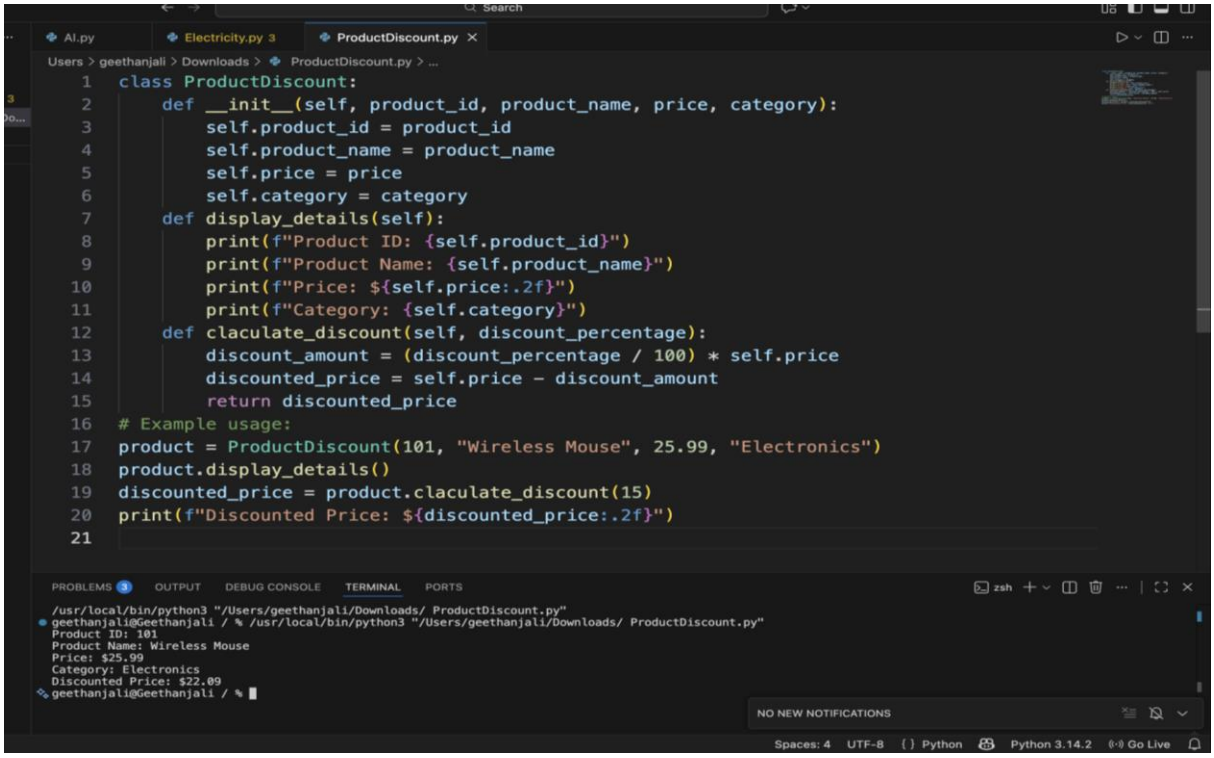
Product Discount Calculation- Create Python code that defines a class named `Product` with attributes: `product_id`, `product_name`,`price`, and `category`. Implement a method `display_details()` to print product details. Implement another method `calculate_discount()` where:

- Electronics → 10% discount

- Clothing → 15% discount

- Grocery → 5% discount

Create at least one product object, display details, and print the final price after discount.



Task 4:

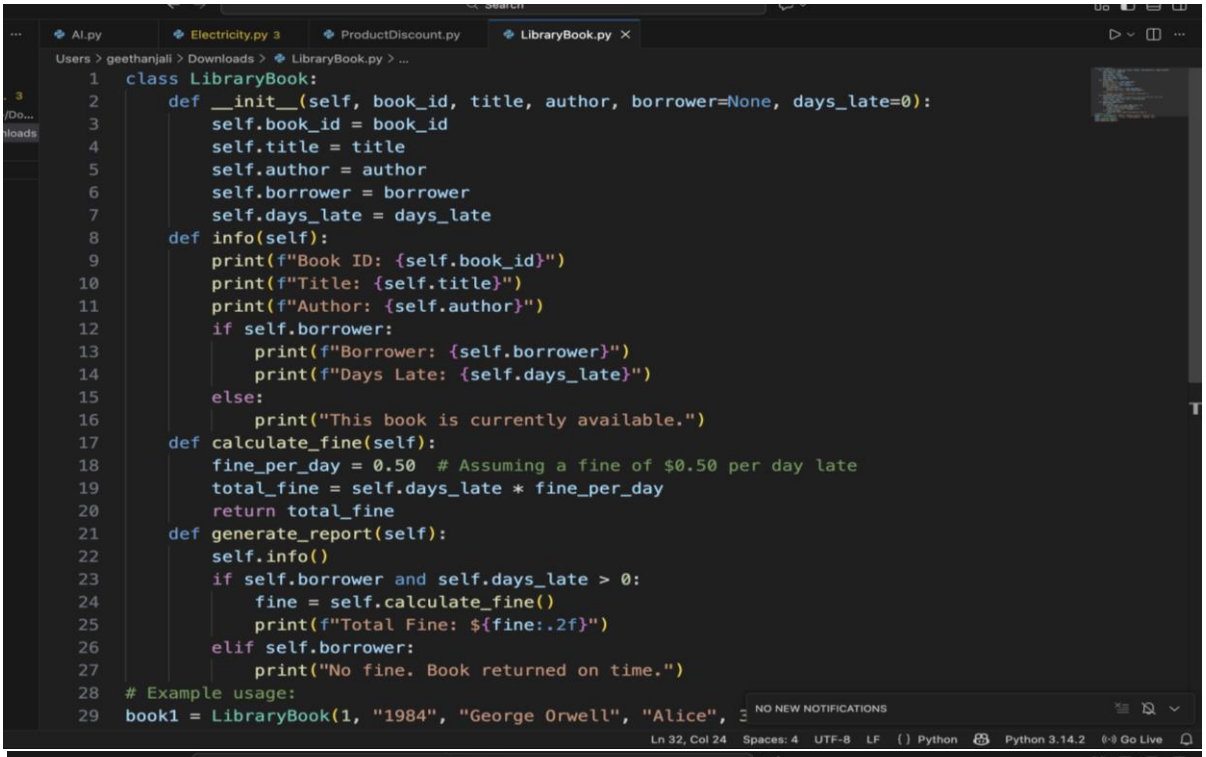
Book Late Fee Calculation- Create Python code that defines a class named `LibraryBook` with attributes: `book_id`, `title`, `author`,`borrower`, and `days_late`. Implement a method `display_details()`to print book details, and a method `calculate_late_fee()` where:

- Days late ≤ 5 → ₹5 per day

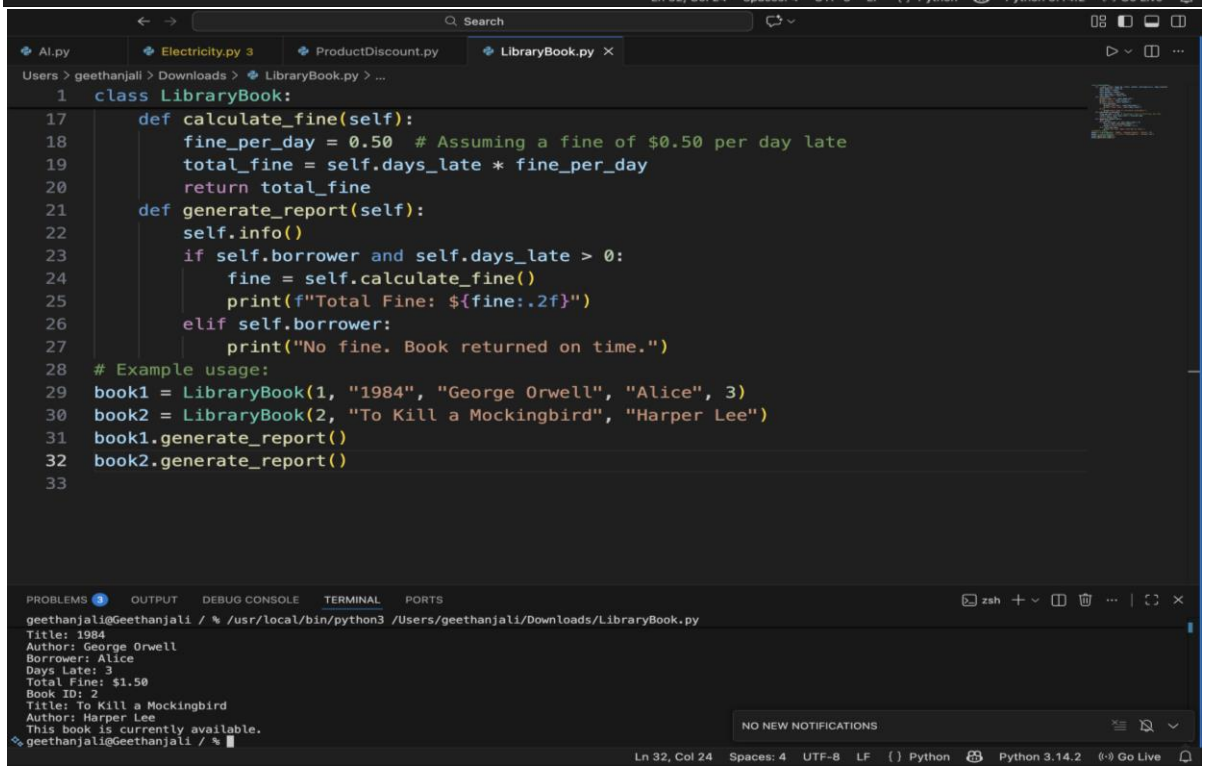
- 6 to 10 days late → ₹7 per day

- More than 10 days late → ₹10 per day

Create a book object, display details, and print the late fee.



```python
class LibraryBook:
    def __init__(self, book_id, title, author, borrower=None, days_late=0):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.borrower = borrower
        self.days_late = days_late
    def info(self):
        print(f"Book ID: {self.book_id}")
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")
        if self.borrower:
            print(f"Borrower: {self.borrower}")
            print(f"Days Late: {self.days_late}")
        else:
            print("This book is currently available.")
    def calculate_fine(self):
        fine_per_day = 0.50  # Assuming a fine of $0.50 per day late
        total_fine = self.days_late * fine_per_day
        return total_fine
    def generate_report(self):
        self.info()
        if self.borrower and self.days_late > 0:
            fine = self.calculate_fine()
            print(f"Total Fine: ${fine:.2f}")
        elif self.borrower:
            print("No fine. Book returned on time.")
# Example usage:
book1 = LibraryBook(1, "1984", "George Orwell", "Alice",
```



```python
class LibraryBook:
    def calculate_fine(self):
        fine_per_day = 0.50  # Assuming a fine of $0.50 per day late
        total_fine = self.days_late * fine_per_day
        return total_fine
    def generate_report(self):
        self.info()
        if self.borrower and self.days_late > 0:
            fine = self.calculate_fine()
            print(f"Total Fine: ${fine:.2f}")
        elif self.borrower:
            print("No fine. Book returned on time.")
# Example usage:
book1 = LibraryBook(1, "1984", "George Orwell", "Alice", 3)
book2 = LibraryBook(2, "To Kill a Mockingbird", "Harper Lee")
book1.generate_report()
book2.generate_report()
```

```
geethanjali@Geethanjali / % /usr/local/bin/python3 /Users/geethanjali/Downloads/LibraryBook.py
Title: 1984
Author: George Orwell
Borrower: Alice
Days Late: 3
Total Fine: $1.50
Book ID: 2
Title: To Kill a Mockingbird
Author: Harper Lee
This book is currently available.
geethanjali@Geethanjali / %
```

Task 5:

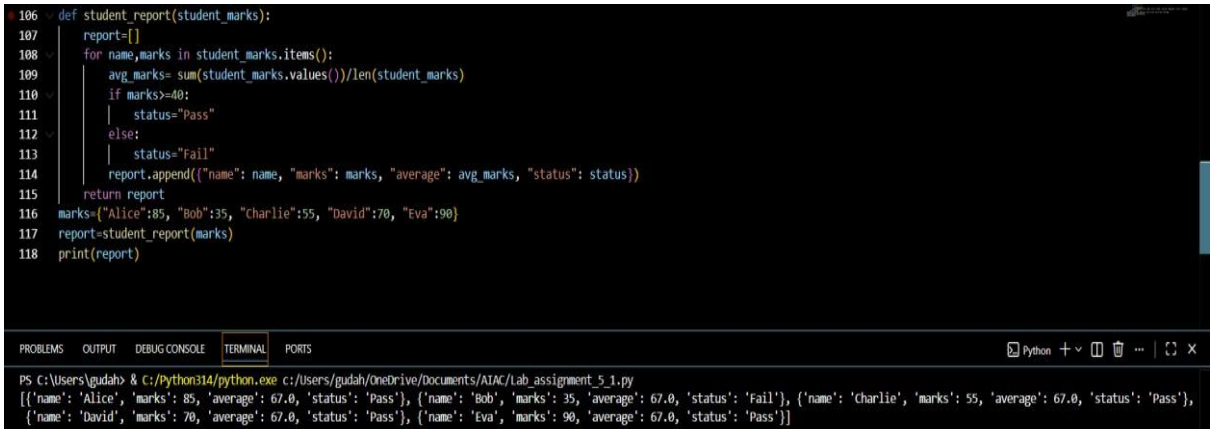
Student Performance Report - Define a function `student_report(student_data)` that accepts a dictionary containing student names and their marks. The function should:

- Calculate the average score for each student

- Determine pass/fail status (pass ≥ 40)

- Return a summary report as a list of dictionaries

Use Copilot suggestions as you build the function and format the output.



```
def student_report(student_marks):
    report=[]
    for name,marks in student_marks.items():
        avg_marks= sum(student_marks.values())/len(student_marks)
        if marks>=40:
            status="Pass"
        else:
            status="Fail"
        report.append({"name": name, "marks": marks, "average": avg_marks, "status": status})
    return report
marks={"Alice":85, "Bob":35, "Charlie":55, "David":70, "Eva":90}
report=student_report(marks)
print(report)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
[{'name': 'Alice', 'marks': 85, 'average': 67.0, 'status': 'Pass'}, {'name': 'Bob', 'marks': 35, 'average': 67.0, 'status': 'Fail'}, {'name': 'Charlie', 'marks': 55, 'average': 67.0, 'status': 'Pass'},
 {'name': 'David', 'marks': 70, 'average': 67.0, 'status': 'Pass'}, {'name': 'Eva', 'marks': 90, 'average': 67.0, 'status': 'Pass'}]
```

Task 6:

Taxi Fare Calculation-Create Python code that defines a class named `TaxiRide` with attributes: `ride_id`, `driver_name`, `distance_km`, and `waiting_time_min`. Implement a method `display_details()` to print ride details, and a method `calculate_fare()` where:

- ₹15 per km for the first 10 km

- ₹12 per km for the next 20 km

- ₹10 per km above 30 km

- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

```python
class TaxiRide:
    def __init__(self,ride_id,driver_name,distance_km,waiting_time_min):
        self.ride_id = ride_id
        self.driver_name = driver_name
        self.distance_km = distance_km
        self.waiting_time_min = waiting_time_min
    def display_details(self):
        print(f"Ride ID: {self.ride_id}")
        print(f"Driver Name: {self.driver_name}")
        print(f"Distance (km): {self.distance_km}")
        print(f"Waiting Time (min): {self.waiting_time_min}")
    def calculate_fare(self):
        if self.distance_km <=10:
            fare = self.distance_km * 15
        elif 11 <= self.distance_km <=30:
            fare = (10 * 15) + (self.distance_km - 10) * 12
        else:
            fare = (10 * 15) + (20 * 12) + (self.distance_km - 30) * 10
        fare += self.waiting_time_min * 2
        return fare
ride = TaxiRide(501, "Charlie Brown", 25, 10)
ride.display_details()
fare = ride.calculate_fare()
print(f"Total Fare: {fare}")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Ride ID: 501
Driver Name: Charlie Brown
Distance (km): 25
Waiting Time (min): 10
Total Fare: 350
```

Task 7:

Statistics Subject Performance - Create a Python function `statistics_subject(scores_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following: - Highest score in the class

- Lowest score in the class

- Class average score

- Number of students passed (score ≥ 40)

- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

```python
def statistics_subject(score_list):
    total = sum(score_list)
    average = total / len(score_list)
    highest = max(score_list)
    lowest = min(score_list)
    passed = 0
    failed=0
    for i in score_list:
        if i>=40:
            passed+=1
        else:
            failed+=1
    print(f"Number of Students Passed: {passed}")
    print(f"Number of Students Failed: {failed}")
    return { "average": average, "highest": highest, "lowest": lowest}
scores = [ 28, 49, 33,  72, 15, 60, 95, 40, 53, 81, 22, 47, 68, 79, 34, 91, 44, 58, 73, 38, 66, 84, 29, 50, 77, 92, 41, 36, 65, 80, 54, 87, 30, 69, 45, 71, 39, 83, 59, 74
stats = statistics_subject(scores)
print(stats)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Number of Students Passed: 46
Number of Students Failed: 14
{'average': 58.6, 'highest': 95, 'lowest': 15}
```

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Task Description #8 (Transparency in Algorithm Optimization)

Task: Use AI to generate two solutions for checking prime numbers:

• Naive approach(basic)

• Optimized approach

Prompt:

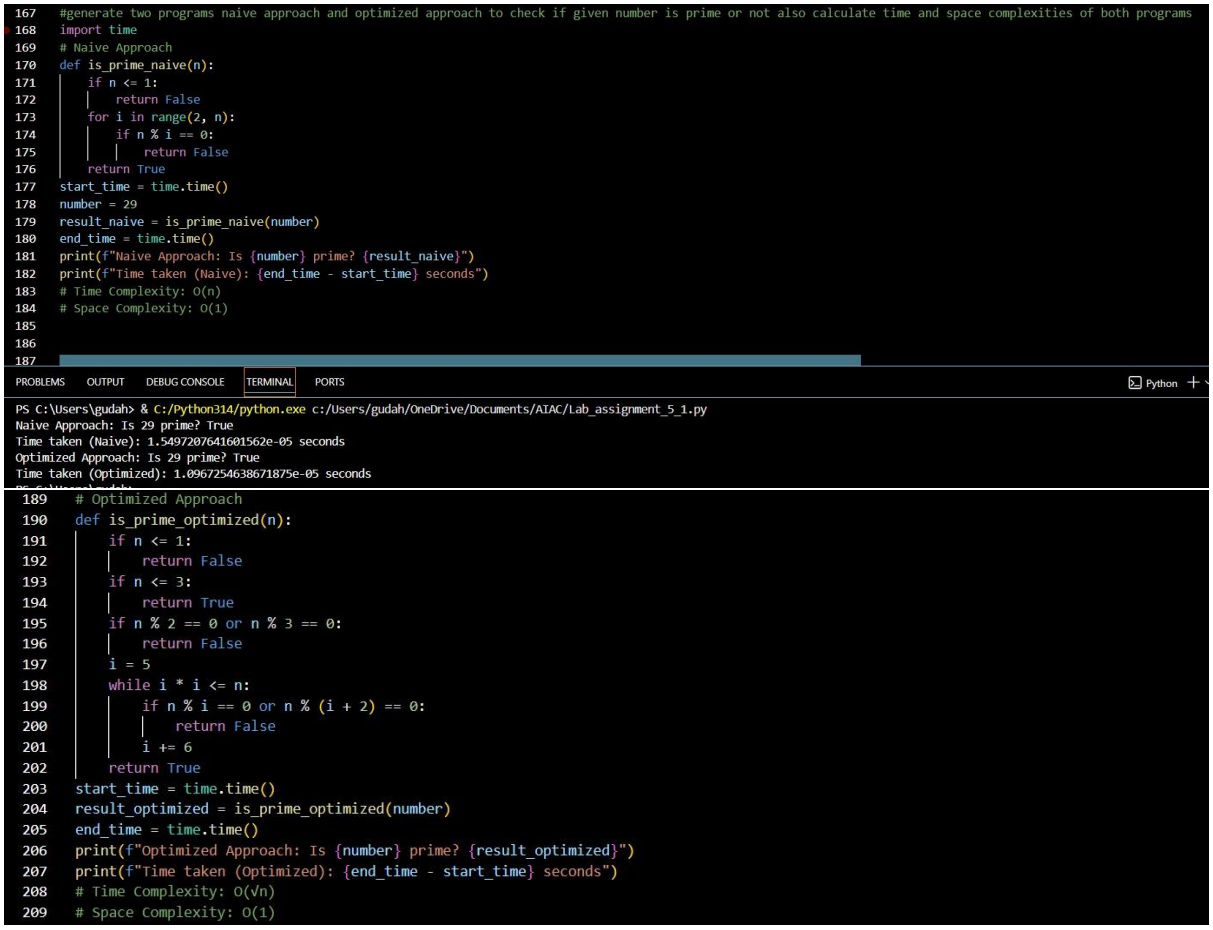
“Generate Python code for two prime-checking methods and explain how the optimized version improves performance.”

Expected Output:

• Code for both methods.

• Transparent explanation of time complexity.

• Comparison highlighting efficiency improvements.



```python
#generate two programs naive approach and optimized approach to check if given number is prime or not also calculate time and space complexities of both programs
import time
# Naive Approach
def is_prime_naive(n):
    if n <= 1:
        return False
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
start_time = time.time()
number = 29
result_naive = is_prime_naive(number)
end_time = time.time()
print(f"Naive Approach: Is {number} prime? {result_naive}")
print(f"Time taken (Naive): {end_time - start_time} seconds")
# Time Complexity: O(n)
# Space Complexity: O(1)
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                                    Python
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Naive Approach: Is 29 prime? True
Time taken (Naive): 1.5497207641601562e-05 seconds
Optimized Approach: Is 29 prime? True
Time taken (Optimized): 1.0967254638671875e-05 seconds
```

```python
# Optimized Approach
def is_prime_optimized(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
start_time = time.time()
result_optimized = is_prime_optimized(number)
end_time = time.time()
print(f"Optimized Approach: Is {number} prime? {result_optimized}")
print(f"Time taken (Optimized): {end_time - start_time} seconds")
# Time Complexity: O(√n)
# Space Complexity: O(1)
```

Task Description #9 (Transparency in Recursive Algorithms)

Objective: Use AI to generate a recursive function to calculate Fibonacci numbers.

Instructions:

1. Ask AI to add clear comments explaining recursion.

2. Ask AI to explain base cases and recursive calls.

Expected Output:

• Well-commented recursive code.

• Clear explanation of how recursion works.

• Verification that explanation matches actual execution.

```
#generate python code that develop fibonacci sequence using recursion
#Provide well commented explanation of each step
def fibonacci(n):
    # Base case: if n is 0 or 1, return n
    if n <= 1:
        return n
    else:
        # Recursive case: return the sum of the two preceding numbers
        return fibonacci(n - 1) + fibonacci(n - 2)
# Number of terms in the Fibonacci sequence
num_terms = 10
print(f"Fibonacci sequence up to {num_terms} terms:")
for i in range(num_terms):
    print(fibonacci(i), end=" ")
# The above code defines a recursive function to generate Fibonacci numbers.
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\gudah> & C:/Python314/python.exe c:/Users/gudah/OneDrive/Documents/AIAC/Lab_assignment_5_1.py
Fibonacci sequence up to 10 terms:
0 1 1 2 3 5 8 13 21 34
```

Task Description #10 (Transparency in Error Handling)

Task: Use AI to generate a Python program that reads a file and processes data.

Prompt:

“Generate code with proper error handling and clear explanations for each exception.”
Expected Output:

• Code with meaningful exception handling.

• Clear comments explaining each error scenario.

• Validation that explanations align with runtime behavior.

```python
#generate a program that reads a file and process the data
#Generate code with proper error handling and clear explanations for each exception.
def read_file(file_path):
    try:
        # Attempt to open the file
        with open(file_path, 'r') as file:
            data = file.read()
            print("File content successfully read.")
            return data
    except FileNotFoundError:
        # Handle the case where the file does not exist
        print(f"Error: The file at {file_path} was not found.")
    except PermissionError:
        # Handle the case where there are permission issues
        print(f"Error: You do not have permission to read the file at {file_path}.")
    except Exception as e:
        # Handle any other exceptions that may occur
        print(f"An unexpected error occurred: {e}")
file_path = 'example.txt'  # Specify the path to your file here
file_content = read_file(file_path)
if file_content:
    print("File Content:")
    print(file_content)
```

```
File content successfully read.
File Content:
Hello Everyone
Welcome to AI Assisted Coding class
Third year second semester
SR University
Lets work with files as part of lab assignment
```